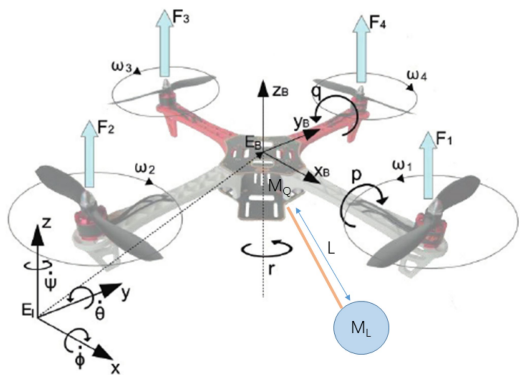


Bridging **Control** with the Modern **Neural Network** **Verifier α, β -CROWN: A Tutorial**

Tutorial Organizers: **Huan Zhang**, Bin Hu (UIUC)

Slides & Code: <https://abcrown.org/tutorial>

Speakers: Huan Zhang, Xiangru Zhong, Haoyu Li



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

Guarantees for learning-enabled & nonlinear controllers

Many traditional tools are used in nonlinear control:

- Sum-of-squares (SOS), Linear Matrix Inequalities (LMIs), Semidefinite programming (SDP), SMT solvers, ...
- These techniques are broadly applicable to many different problems
- Although possible, it is challenging when dealing with neural network controlled systems

Call for **new tools** for **learning-enabled** / **nonlinear** control systems

- Efficiency and scalability of handling neural networks (NNs) and other nonlinear components
- General enough; potential to handle many different types of problems
- Easy to use

Today's tutorial: α, β -CROWN Neural Network Verification framework as a new tool

And why α, β -CROWN?

Originally developed for scalable NN verification; the CROWN-family algorithms have become a versatile tool to study of many control problems

Scalable Data-Driven Reachability Analysis and Control via Koopman Operators with Conformal Coverage Guarantees

Devesh Nath^{*1}
Haoran Yin^{*1}
Glen Chou²

Georgia Institute of Technology. ¹ School of ECE, ² Schools of Cybersecurity & Privacy & Aerospace Eng.

L4DC 2026 Best
paper finalist

DNATH7@GATECH.EDU
HYIN95@GATECH.EDU
CHOU@GATECH.EDU

GPU-Accelerated Verification of Machine Learning Models for Power Systems

Best Paper Award at
HICSS 2024

Samuel Chevalier, Ilgiz Murzakanov, and Spyros Chatzivasileiadis
Department of Wind and Energy Systems
Lyngby, Denmark
fcheval@dtu.dk, imurzaka@dtu.dk

2025 IEEE 64th Conference on Decision and Control (CDC)
December 10-12, 2025. Rio de Janeiro, Brazil

CDC 2025

Learning Verifiable Control Policies Using Relaxed Verification

Puja Chaudhury, Alexander Estornell, Michael Everett

Interval Reachability of Nonlinear Dynamical Systems with Neural Network Controllers

Saber Jafarpour^{*}
Georgia Institute of Technology
Akash Harapanahalli^{*}
Georgia Institute of Technology

SABER@GATECH.EDU

AHARAPAN@GATECH.EDU

Samuel Coogan
Georgia Institute of Technology

SAM.COOGAN@GATECH.EDU

L4DC 2023

Scalable Synthesis of Formally Verified Neural Value Function for Hamilton-Jacobi Reachability Analysis

YUJIE YANG, Tsinghua University, China and Carnegie Mellon University, USA
HANJIANG HU, Carnegie Mellon University, USA
TIANHAO WEI, Carnegie Mellon University, USA
SHENGBO EBEN LI, Tsinghua University, China
CHANGLIU LIU^{*}, Carnegie Mellon University, USA

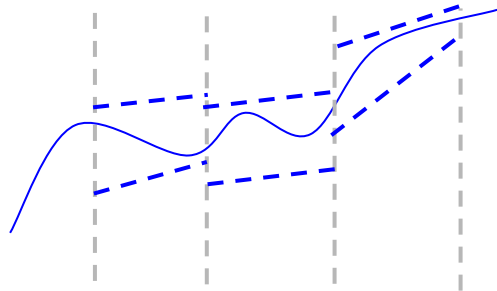
JAIR 2025

ICCPS 2025

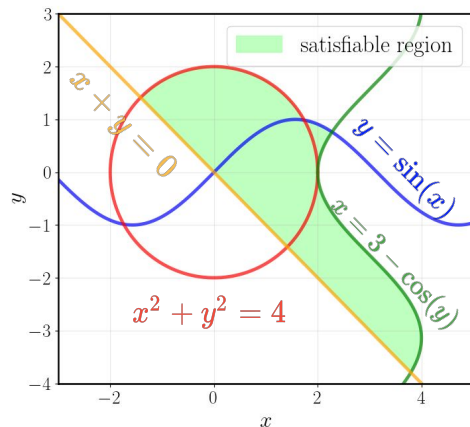
Yue Wang
yue6@clemson.edu
Clemson University
Clemson, South Carolina, USA

Aditya Parameshwaran
aparam@clmson.edu
Clemson University
Clemson, South Carolina, USA

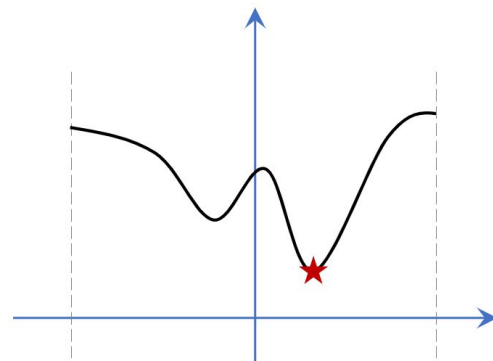
What α, β -CROWN can do for nonlinear functions?



Bounding nonlinear functions



Satisfiability problems with nonlinear arithmetic



Global nonlinear optimization

With a NEW, easy-to-use Python API to do all of these!

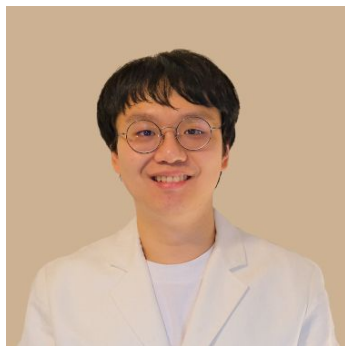
Agenda and Speakers

Part I: Theory and Algorithms of α, β -CROWN (Huan Zhang)

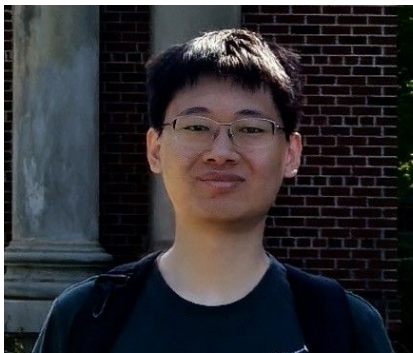
Interactive Coding demo: Basic usage of α, β -CROWN verifier (Xiangru Zhong)

Part II: Application of α, β -CROWN to control problems (Haoyu Li)

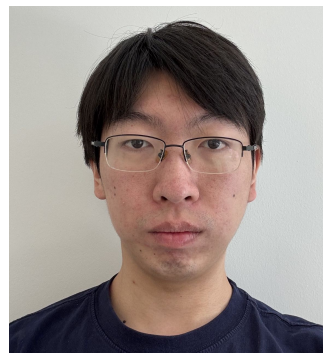
Interactive Coding demo: Examples of α, β -CROWN in control (Xiangru Zhong)



Huan Zhang



Xiangru Zhong



Haoyu Li

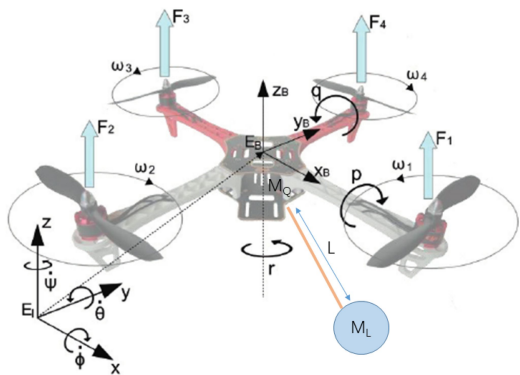
Bridging **Control** with the Modern **Neural Network** **Verifier α,β -CROWN: A Tutorial**

Part I: Theory and Algorithms of α,β -CROWN

Huan Zhang (UIUC)

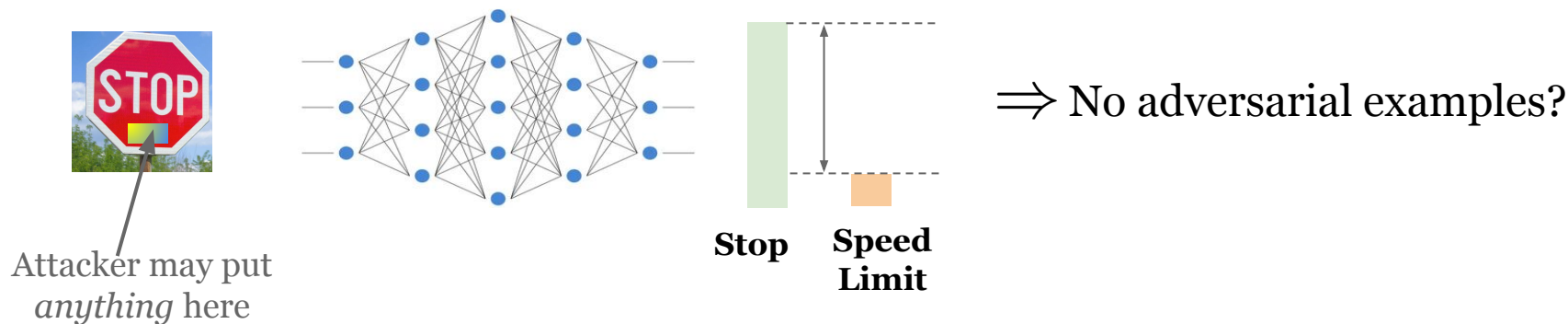
Assistant Professor, Department of Electrical and Computer Engineering

huan@huan-zhang.com



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

From NN verification to nonlinear control problems



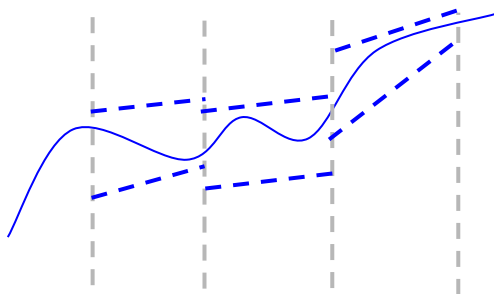
Neural network verification is originally for formally guaranteeing the safety and robustness of NN classifiers (since 2017)

- **Scalable** to NNs with millions of parameters
- **GPU**-accelerated, orders of magnitudes faster
- But **restricted** to a small class of problems

From NN verification to nonlinear control problems

Today, we use our NN verifier (α,β -CROWN) to solve some control problems!

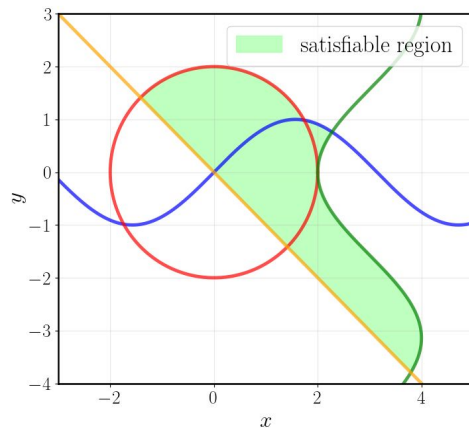
- Give guaranteed **linear bounds** on **complex nonlinear functions**
(can be much *tighter* than Lipschitz analysis!)



From NN verification to nonlinear control problems

Today, we use our NN verifier (α,β -CROWN) to solve some control problems!

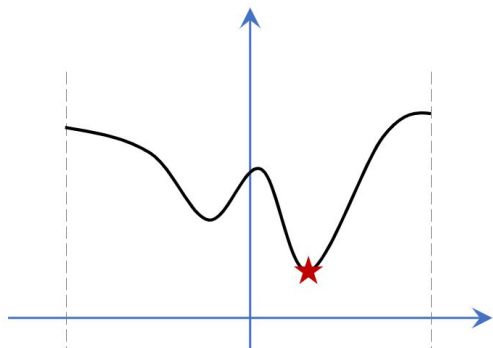
- Give guaranteed **linear bounds** on **complex nonlinear functions** (can be much *tighter* than Lipschitz analysis!)
- Check whether a bunch of **nonlinear inequalities** can be **satisfied** (much *faster* than a SMT solver)



From NN verification to nonlinear control problems

Today, we use our NN verifier (α,β -CROWN) to solve some control problems!

- Give guaranteed **linear bounds** on **complex nonlinear functions** (can be much *tighter* than Lipschitz analysis!)
- Check whether a bunch of **nonlinear inequalities** can be **satisfied** (much *faster* than a SMT solver)
- Solve some **nonlinear optimization problems** in control (your next-gen, GPU-accelerated Gurobi?)



From NN verification to nonlinear control problems

Today, we use our NN verifier (α,β -CROWN) to solve some control problems!

- Give guaranteed **linear bounds** on **complex nonlinear functions**
(can be much *tighter* than Lipschitz analysis!)
- Check whether a bunch of **nonlinear inequalities** can be **satisfied**
(much *faster* than a SMT solver)
- Solve some **nonlinear optimization problems** in control
(your next-gen, GPU-accelerated Gurobi?)

Key characteristics:

- Compose complex nonlinear functions with simple ones
- A neural network is **NOT** required
- Use **GPU** acceleration, great **scalability**



The (very basic) verification problem

$$f(x)$$

1. A nonlinear function f
 - Can be just the NN itself (e.g., a binary classifier)
 - Can also include NN as part of f (e.g., NN-controlled systems)
 - Can include other nonlinear functions (e.g., dynamical systems + NN)
 - You can compose any supported nonlinear functions (NN is not required), as a computation graph

The (very basic) verification problem

$$\forall x \in \mathcal{S}, f(x)$$

1. A nonlinear function f
2. A input set $\mathcal{S}$
 - Simple box constraints (coordinate-wise bounds)
 - Convex set (e.g., L_p -norm ball, convex polytopes...)
 - Or non-convex set (e.g., implicitly defined by other constraints)

The (very basic) verification problem

$$\forall x \in \mathcal{S}, f(x) > 0$$

1. A nonlinear function f
2. A input set S
3. An output specification
 - Simply $f(x) > 0$
 - Complex logic formulas $(f(x) > 0 \vee g(x) > 0) \wedge (h(x) > 0)$

The (very basic) verification problem

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

(this is a “simple” verification problem in canonical form)

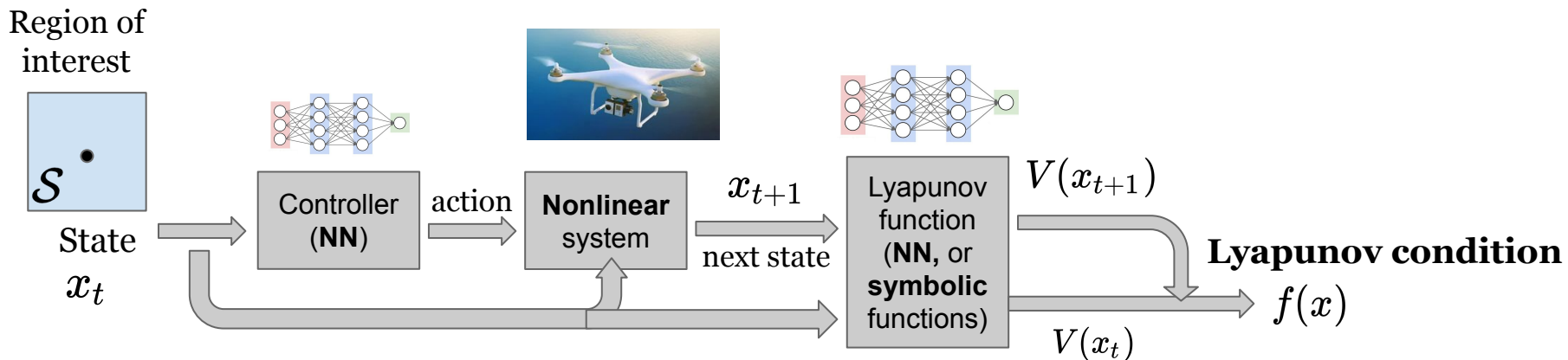
1. A nonlinear function f
2. A input set S
3. An output specification

α, β -CROWN can prove problems like this for you, automatically 

But how can this help control problems ?

Example: verification of the stability of NN controllers

More examples and details in the second part of this tutorial



$$f(x) := V(x_t) - V(x_{t+1}) > 0$$

$f(x)$ is a “computation graph”, representing the algebraic condition you want to verify

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

(other conditions on V can be satisfied by a special construction of V)

Some comparisons to existing tools

Task: Lyapunov stability verification with α, β -CROWN
NN controller + NN Lyapunov function (a few hundreds neurons)

System	dReal	α, β -CROWN
Van der Pol	39265.21s	3.94s
Double Integrator	359.27s	3.00s
Pendulum (large torque)	1479.55s	3.64s
Pendulum (small torque)	1709.19s	3.94s
Path Tracking (large torque)	113.72s	3.77s
Path Tracking (small torque)	142.20s	3.67s

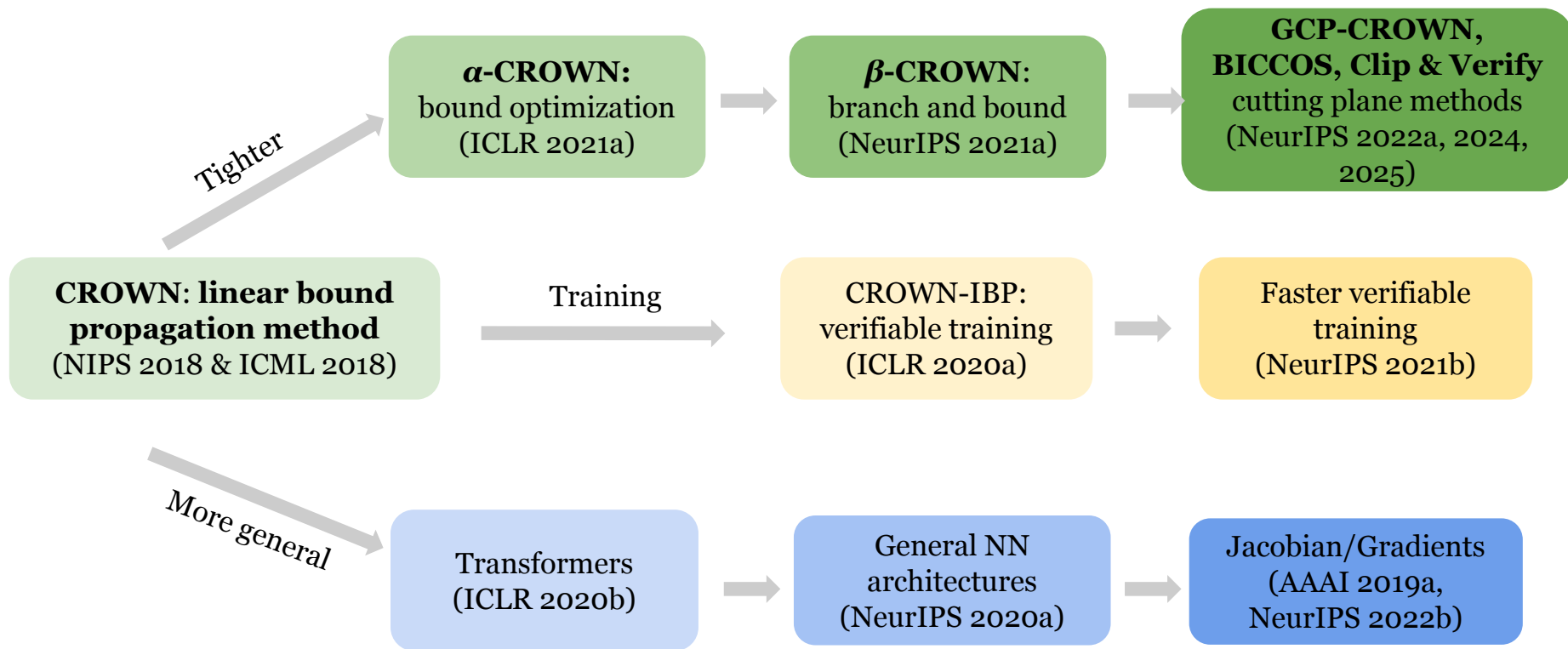
How do we
achieve this?

“Two-Stage Learning of Stabilizing Neural Controllers via Zubov Sampling and Iterative Domain Expansion.” (**Continuous-Time stability analysis**)

Haoyu Li*, Xiangru Zhong*, Bin Hu, Huan Zhang

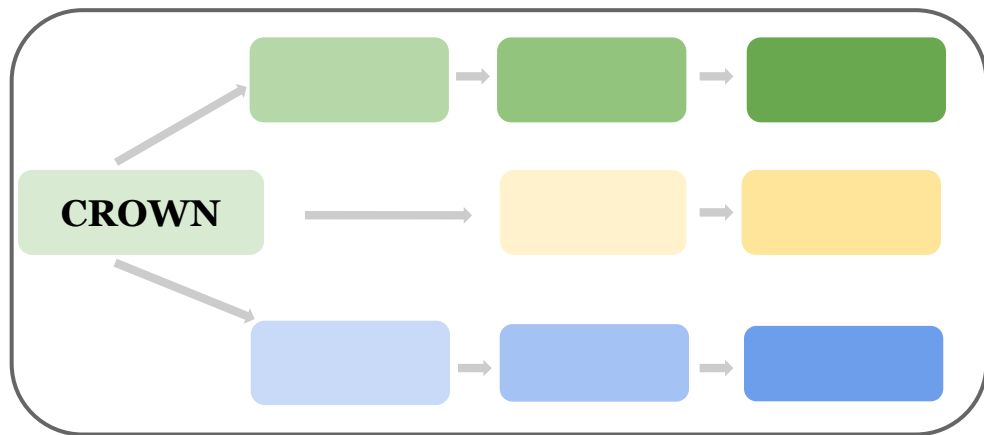
Paper: <https://arxiv.org/pdf/2506.01356>

Years of efforts to strengthen the framework



Algorithms to the practical tool: α, β -CROWN

Theoretical framework



Practical verifier: α, β -CROWN



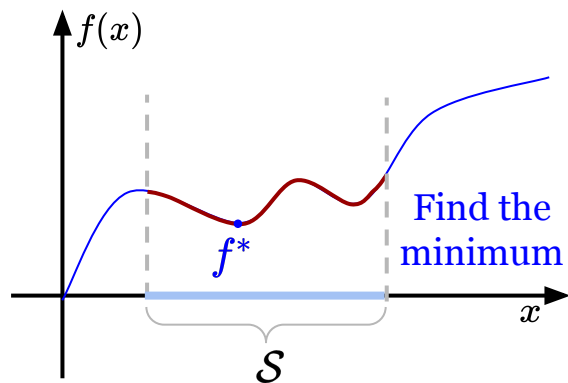
<https://abcrown.org>

Winner of VNN-COMP 2021, 2022, 2023, 2024, 2025

Some existing work may refer to or use **auto_LiRPA**, part of α, β -CROWN; directly using the α, β -CROWN Python API is now the recommended way

How does α, β -CROWN solve the verification problem?

Prove: $\forall x \in \mathcal{S}, f(x) > 0$



Challenges:

Number of variables $\approx$ number of neurons

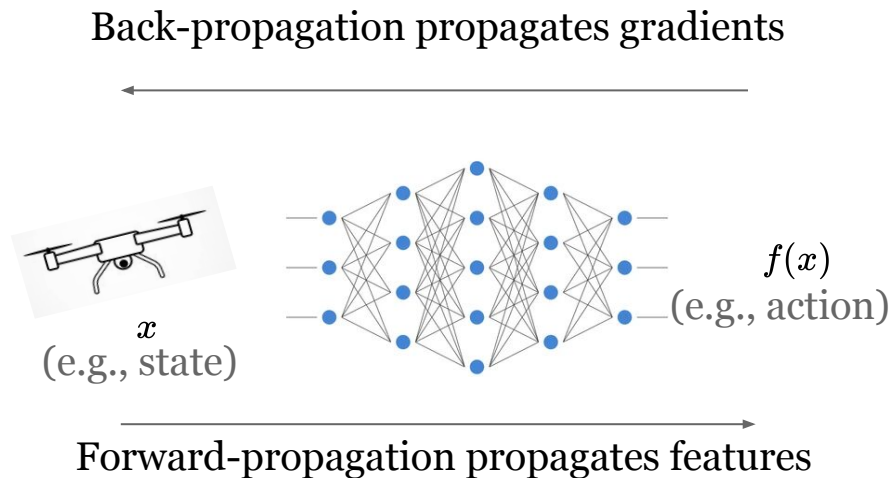
Number of integer variables $\approx$ number of nonlinear functions (e.g., ReLU)

Modern NNs can have **millions** of neurons!

Existing approaches: Linear programming (LP), semidefinite programming (SDP), mixed integer programming (MIP), Satisfiability Modulo Theory (SMT)



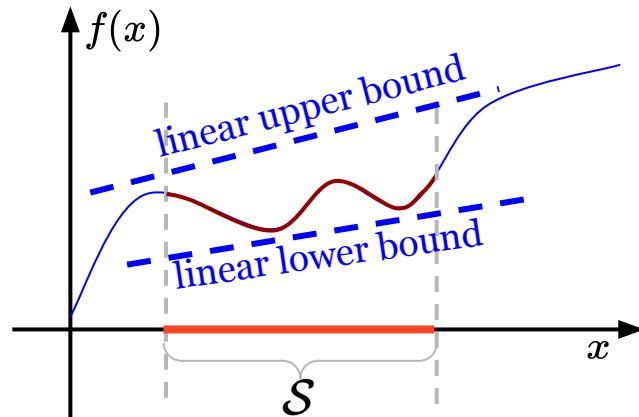
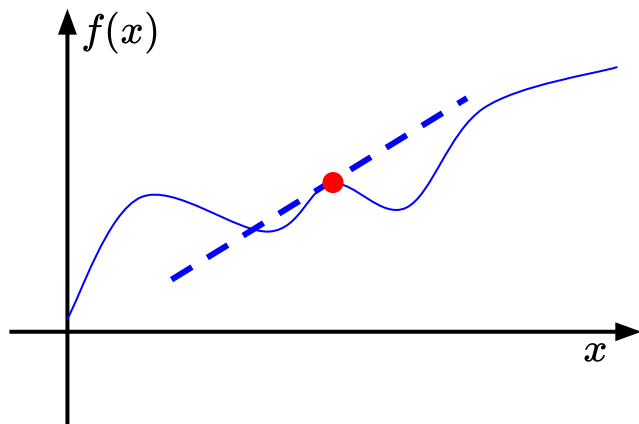
We need to exploit the problem structure to be fast!



Inspiration: propagating things on NNs and a computation graph is **fast**!

Using off-the-shelf solvers (LP/SDP/MIP/SMT) is **slow**!
We don't want use these; we need special, parallelizable algorithms

Key insight in the α, β -CROWN framework: generalize **backpropagation** to **linear bound propagation**



Gradient is neither an upper nor a lower bound for a non-convex function

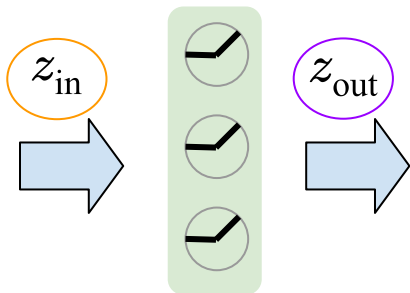
Gradient can be calculated very efficiently (chain rule & backpropagation)

Can we use linear functions to locally bound nonlinear functions?

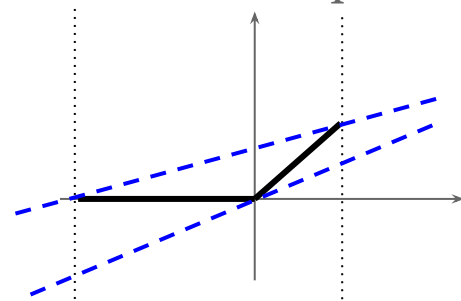
What is the **new “chain rule”** for propagating bounds?

CROWN linear relaxations

Key idea: use linear bounds to relax a non-linear function. Propagation of linear inequalities over relaxed nonlinear functions yields new linear inequalities



$$z_{\text{out}} = \text{ReLU}(z_{\text{in}})$$



Pre-activation bounds
(can be pre-computed using CROWN)

Theorem (informal): we can efficiently find D , b' such that:

$$f(x) \geq a^\top D z_{\text{in}} + b' \longleftarrow f(x) \geq a^\top z_{\text{out}} + b \quad \forall x \in \mathcal{S}$$

CROWN linear relaxations

Proof sketch: conservatively use linear bounds to replace a non-linear function.

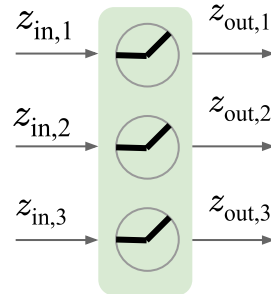
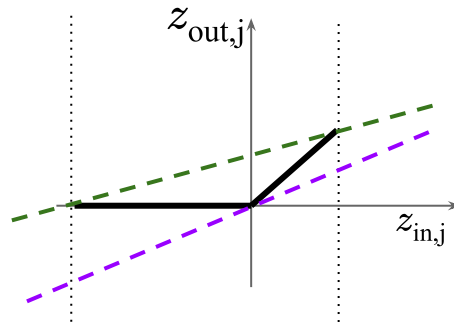
$$f(x) \geq a^\top z_{\text{out}} + b$$

$$\Rightarrow f(x) \geq \sum_j (a_j \cdot z_{\text{out},j}) + b$$

Lower bound each term individually

$a_j < 0$ Plug in linear upper bound

$a_j \geq 0$ Plug in linear lower bound



CROWN linear relaxations

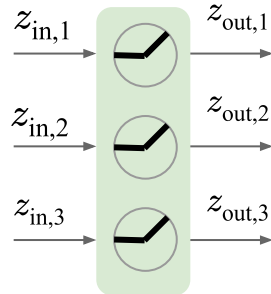
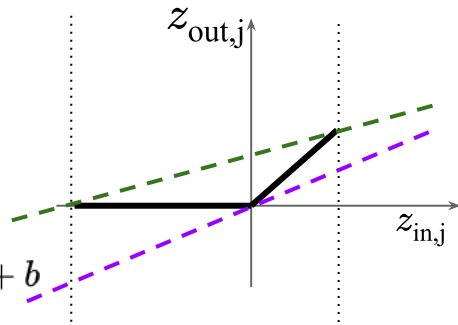
Proof sketch: conservatively use linear bounds to replace a non-linear function.

$$f(x) \geq a^\top z_{\text{out}} + b$$

$$\Rightarrow f(x) \geq \sum_j (a_j \cdot z_{\text{out},j}) + b$$



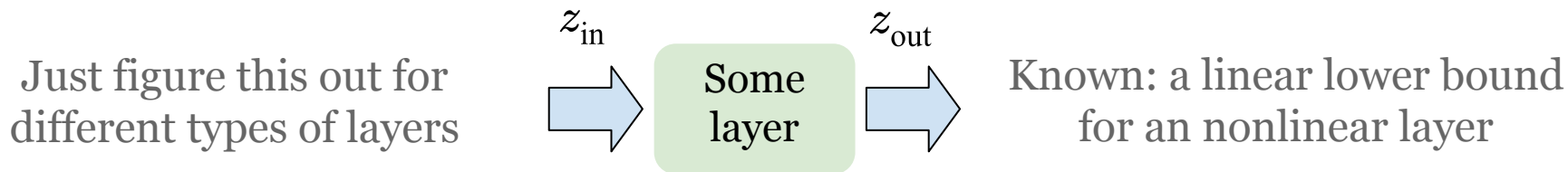
$$f(x) \geq \sum_{a_j > 0} (a_j \times \text{linear func. of } z_{\text{in},j}) + \sum_{a_j \leq 0} (a_j \times \text{linear func. of } z_{\text{in},j}) + b$$



Theorem (informal): we can efficiently find D , b' such that:

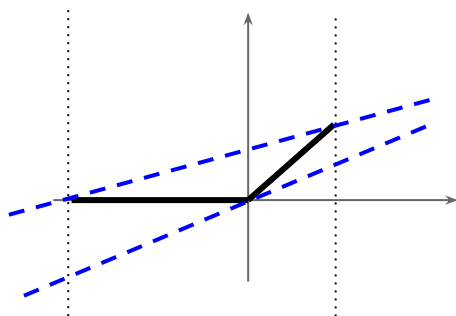
$$f(x) \geq a^\top D z_{\text{in}} + b' \longleftarrow f(x) \geq a^\top z_{\text{out}} + b \quad \forall x \in \mathcal{S}$$

Figure out how to propagate through simple nonlinear functions!

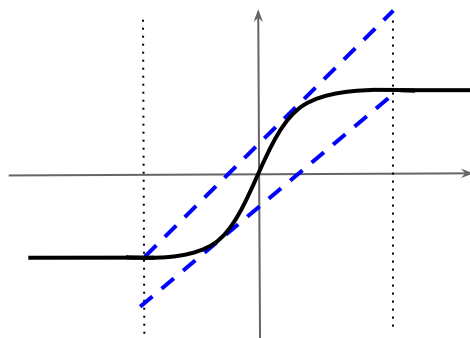
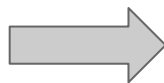


$$f(x) \geq \boxed{a'^{\top}} z_{\text{in}} + \boxed{b'} \quad \longleftarrow \quad f(x) \geq \boxed{a^{\top}} z_{\text{out}} + \boxed{b}$$

Like backpropagation, but we propagate linear inequalities



ReLU

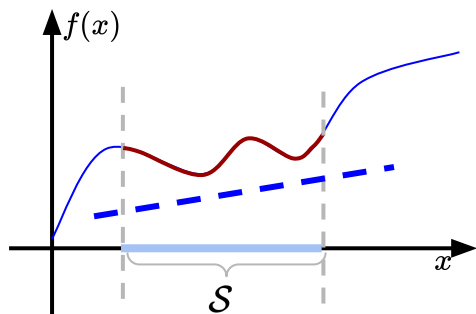


Same trick works for other nonlinear functions

$x^2, x^3, \sqrt{x}, \log x, e^x, |x|, x \cdot y$
 $\sin, \cos, \tan, \tanh, \arctan, \dots$

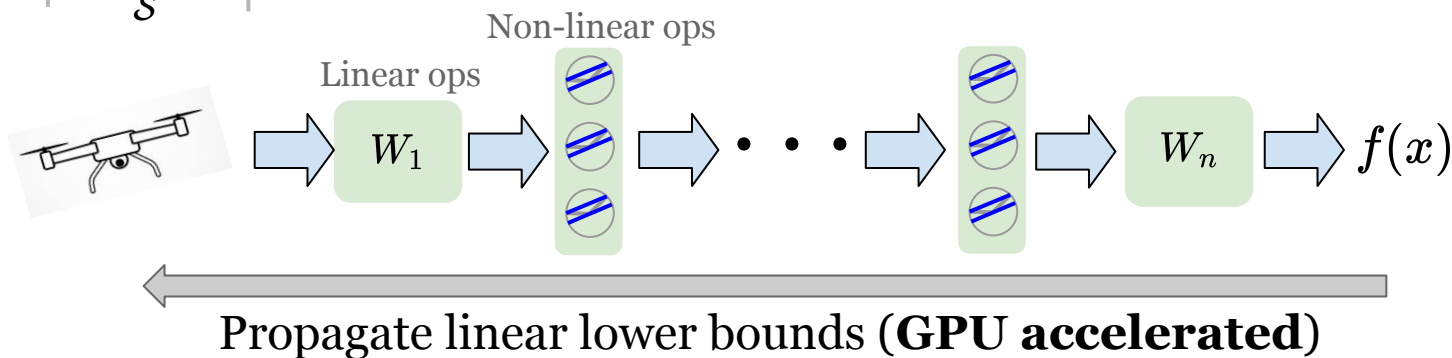
A local linear bound is
all you need

CROWN: a linear bound propagation algorithm



Prove: $\forall x \in \mathcal{S}, f(x) > 0$

Lower bound $> 0 \Rightarrow f(x) > 0 \Rightarrow$ verified

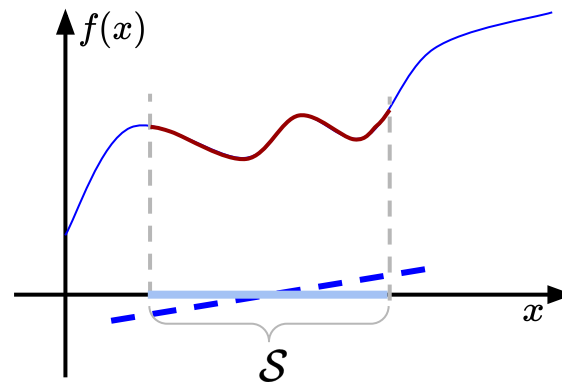
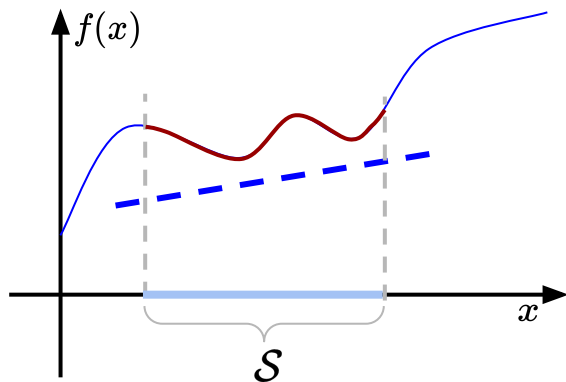


CROWN main theorem (simplified): $f(x) \geq a_{\text{CROWN}}^\top x + b_{\text{CROWN}} \quad \forall x \in \mathcal{S}$

Found by construction (linear bound propagation)

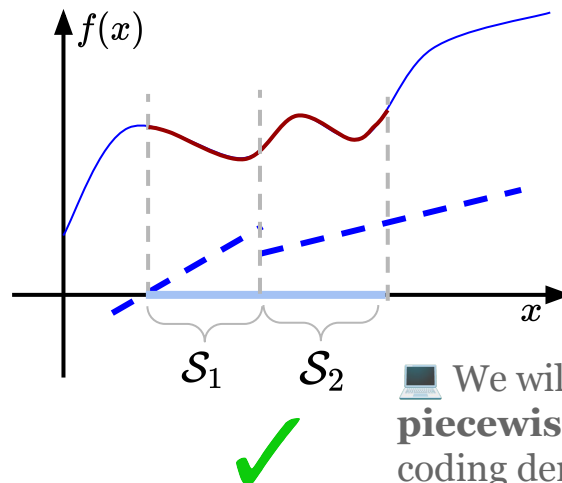
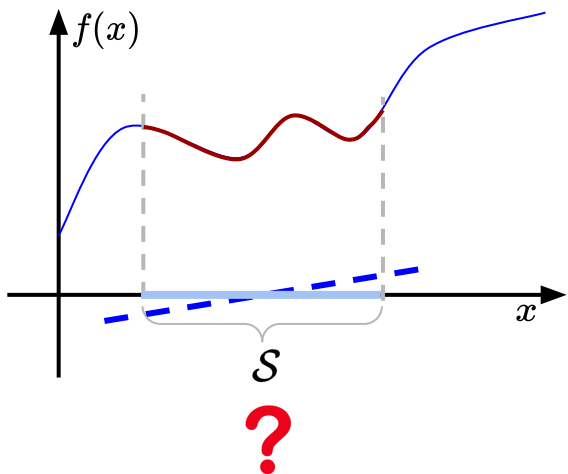
What if the bounds are too loose?

Prove: $\forall x \in \mathcal{S}, f(x) > 0$



Branch and bound to improve tightness

- Branching the original problems into multiple subproblems, each one on a smaller domain with a tighter bound
- Original problem is solved when all subproblems are solved
- α, β -CROWN solves many (e.g., 10,000) subproblems on GPUs in parallel

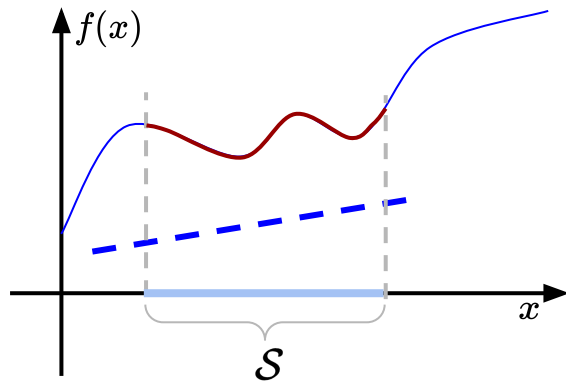


 We will show you these **piecewise linear bound** in coding demo

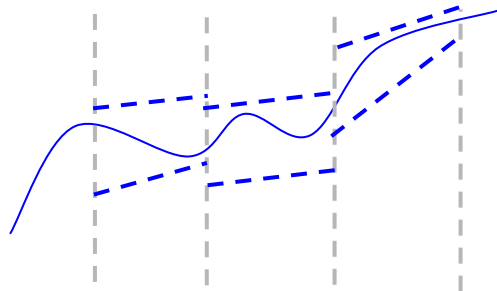
Limitations to keep in mind

- CROWN bounds are local, typically require a bounded set on inputs
- Easiest way is to assume the inputs x are coordinate-wise bounded (box constraint)
- Non-box constraints (e.g., $g(x) > 0$ where $g(x)$ is another nonlinear function) can also be used **in addition to** box constraint

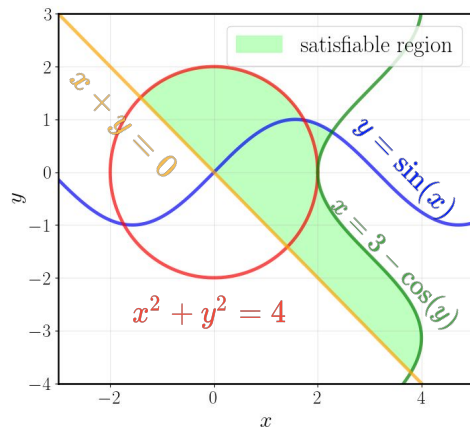
Prove: $\forall x \in \mathcal{S}, f(x) > 0$



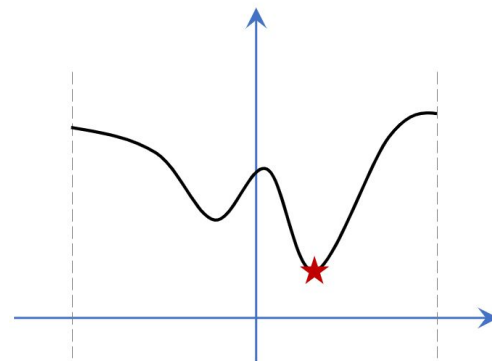
What α, β -CROWN can do for us?



Bounding nonlinear functions



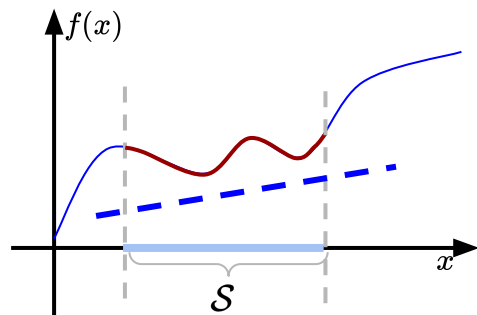
Satisfiability problems with
nonlinear arithmetic



Global nonlinear optimization

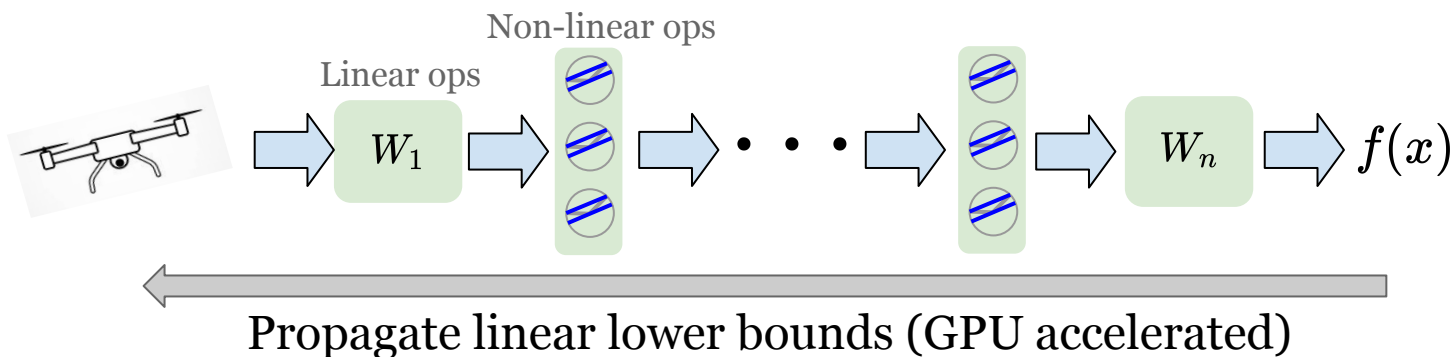


We can use the lower bound to prove $f(x) > 0$



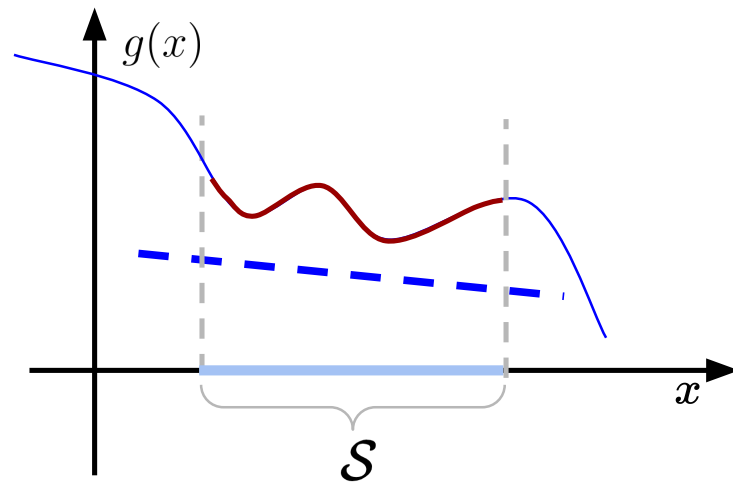
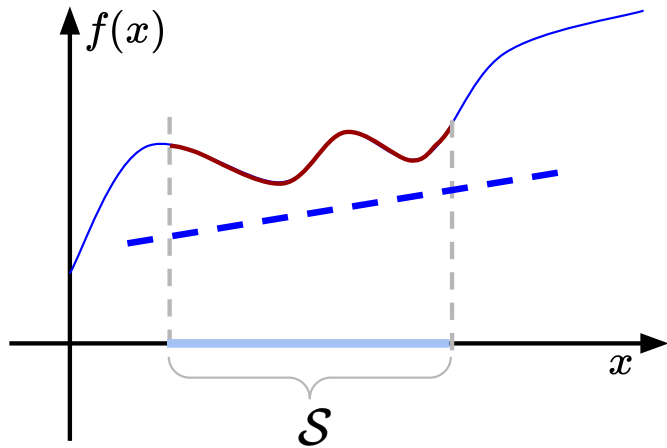
Prove: $\forall x \in \mathcal{S}, f(x) > 0$

Lower bound $> 0 \Rightarrow f(x) > 0 \Rightarrow$ verified



More complicated problems?

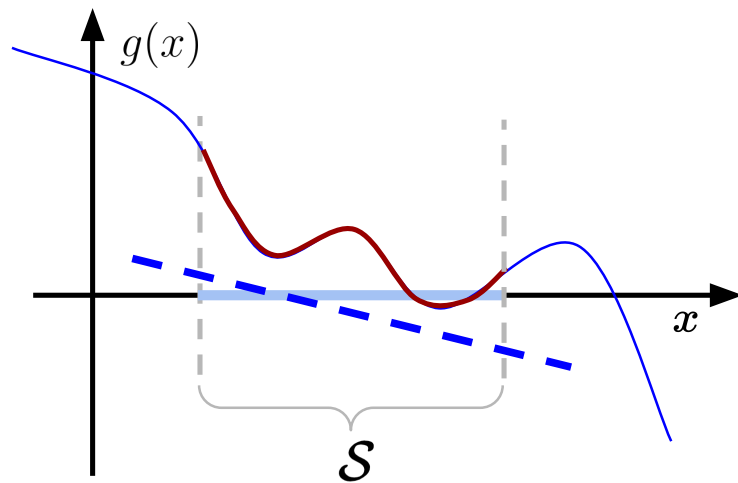
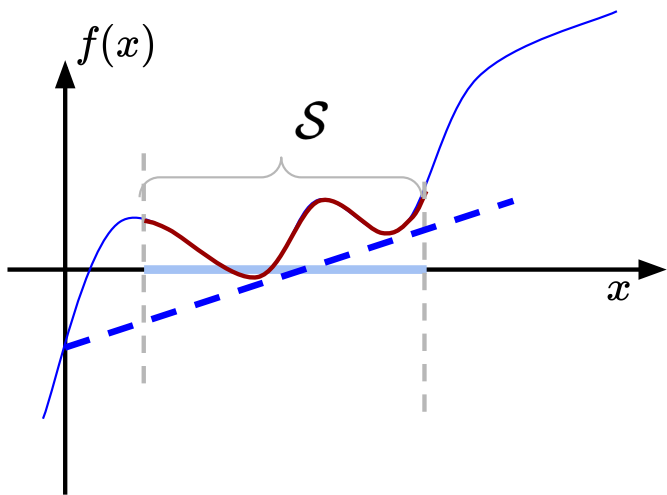
How about proving $\forall x \in \mathcal{S}, f(x) > 0 \wedge g(x) > 0$



Quite simple... just check the linear bounds on both functions

More complicated problems?

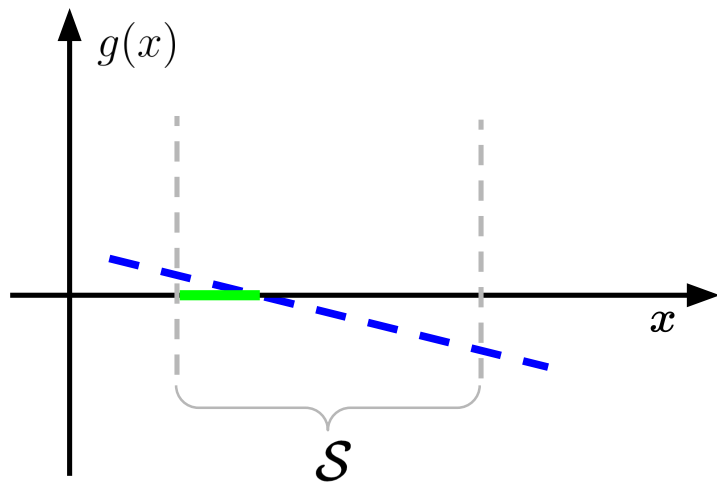
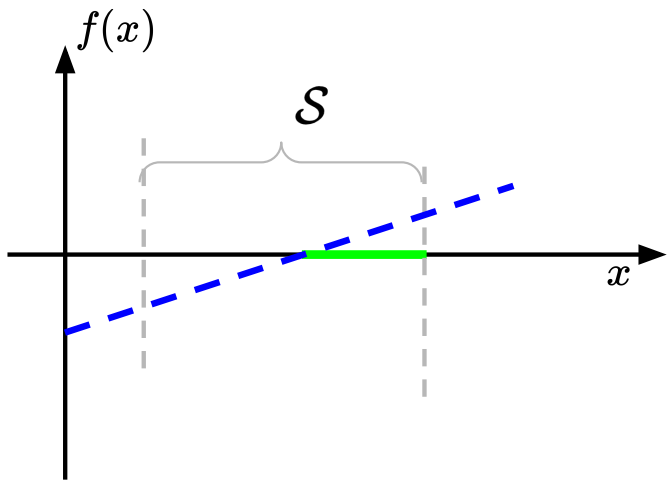
How about proving $\forall x \in \mathcal{S}, f(x) > 0 \quad \vee \quad g(x) > 0$



Just using linear bounds are inconclusive...

More complicated problems?

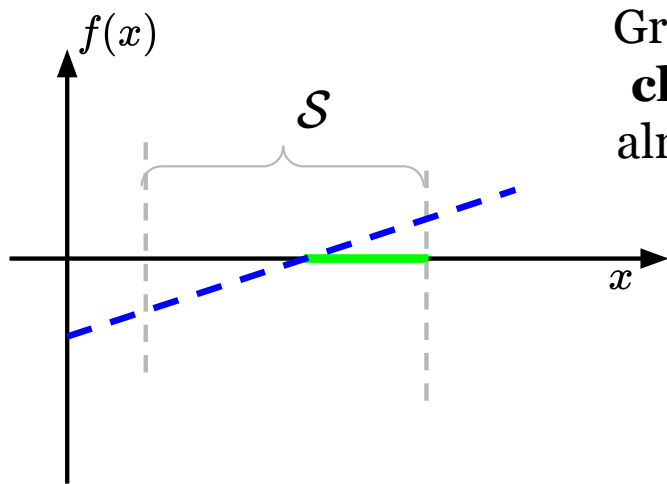
How about proving $\forall x \in \mathcal{S}, f(x) > 0 \vee g(x) > 0$



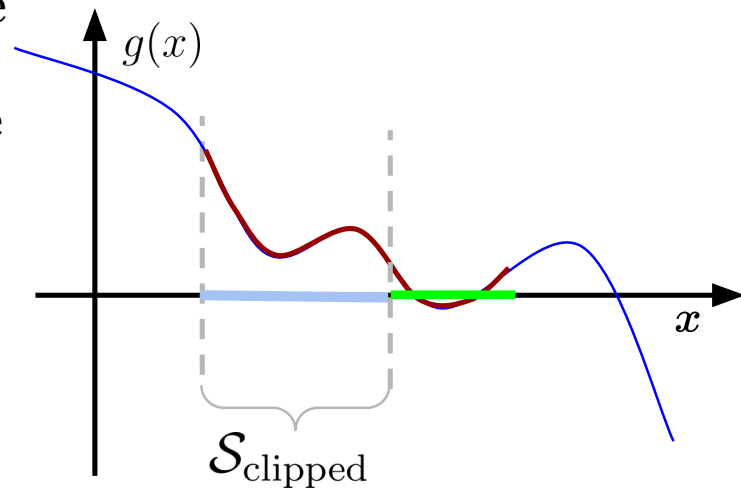
The union of the two partially verified region does not fully contain $\mathcal{S}$

The clip-and-verify procedure

How about proving $\forall x \in \mathcal{S}, f(x) > 0 \vee g(x) > 0$



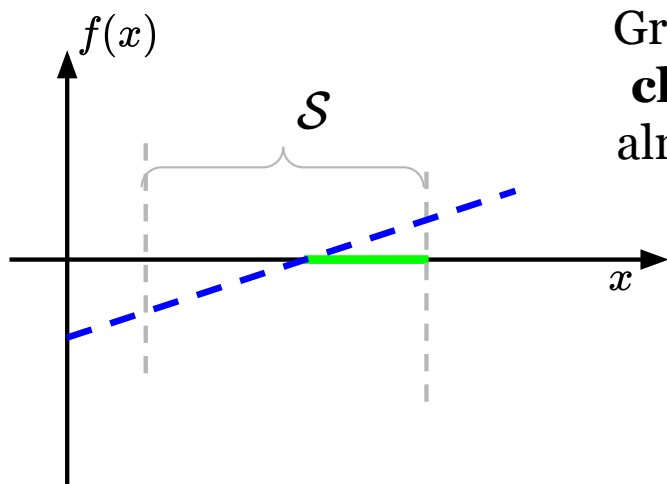
Green region can be
clipped since $f(x)$
already satisfies the
condition!



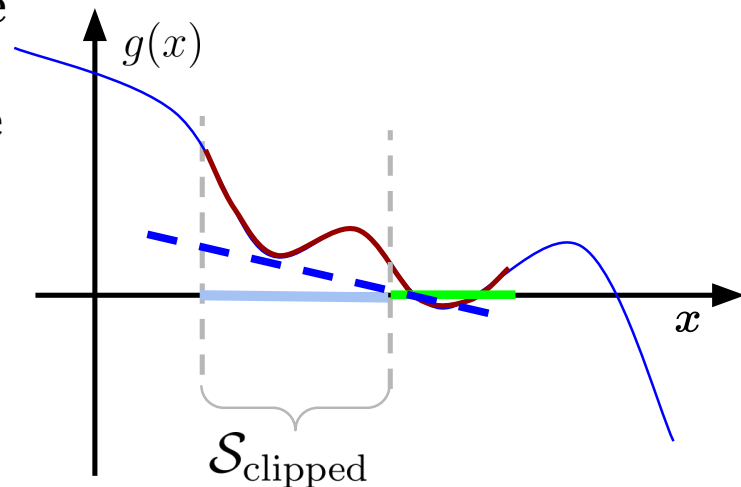
But can we utilize these partially verified regions?

The clip-and-verify procedure

How about proving $\forall x \in \mathcal{S}, f(x) > 0 \quad \vee \quad g(x) > 0$



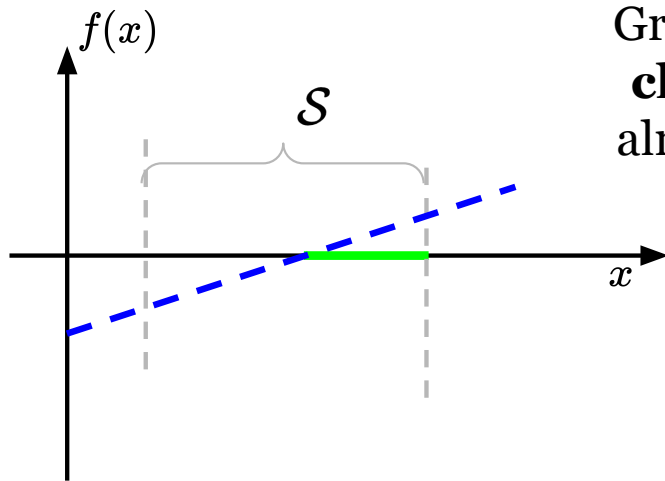
Green region can be
clipped since $f(x)$
already satisfies the
condition!



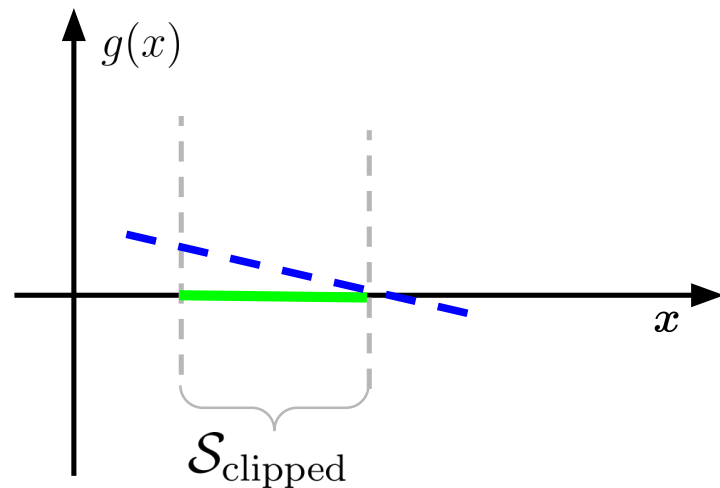
Clipping leads to tighter bounds on $g(x)$ due to the smaller set considered

The clip-and-verify procedure

How about proving $\forall x \in \mathcal{S}, f(x) > 0 \vee g(x) > 0$



Green region can be
clipped since $f(x)$
already satisfies the
condition!



Now the property is proved!

Clip-and-Verify: Linear Constraint-Driven Domain Clipping for Accelerating Neural Network Verification. D. Zhou, J. Chavez, H. Chen, G.A. Hanasusanto, H. Zhang. (NeurIPS 2025)

Huan Zhang (UIUC)

Solving general satisfiability problems

We know how to handle AND and OR conditions. For general satisfiability problem, just convert them into a conjunctive normal form (CNF):

$$\forall x \in \mathcal{S}, \bigwedge_{i=1}^m \bigvee_{j=1}^{n_i} f_{ij}(x) > 0$$

Verify the m AND clauses one by one, each one contains multiple ORs



We will show you how to write control problems in this form (after the break)

Solving general satisfiability problems

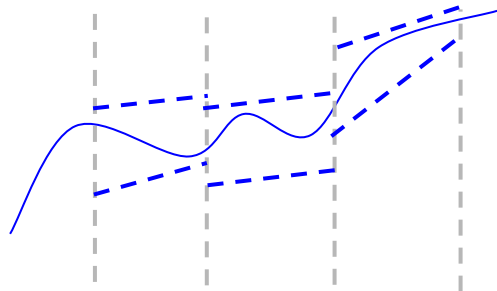
We know how to handle AND and OR conditions. For general satisfiability problem, just convert them into a conjunctive normal form (CNF):

$$\forall x \in \mathcal{S}, \bigwedge_{i=1}^m \bigvee_{j=1}^{n_i} f_{ij}(x) > 0 \quad \text{“Prove safety”}$$

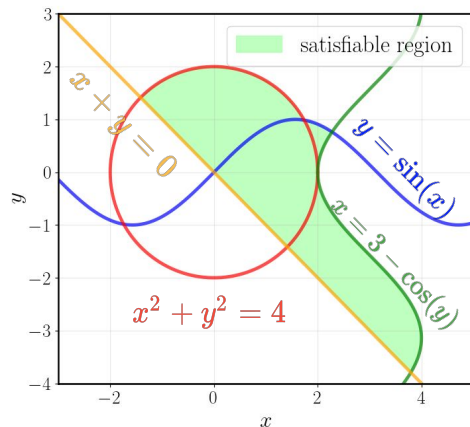
Its negation is equivalent to the canonical satisfiability problem in disjunctive normal form (DNF):

$$\exists x \in \mathcal{S}, \bigvee_{i=1}^m \bigwedge_{j=1}^{n_i} f_{ij}(x) \leq 0 \quad \text{“Find counterexamples”}$$

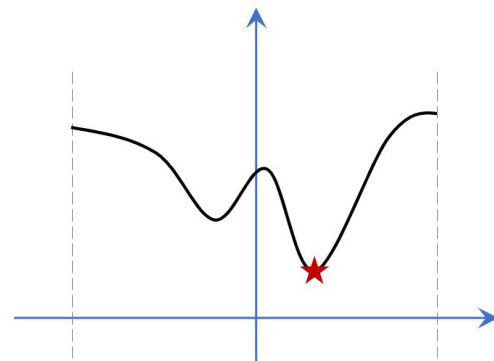
What α, β -CROWN can do for us?



Bounding nonlinear functions



Satisfiability problems with
nonlinear arithmetic



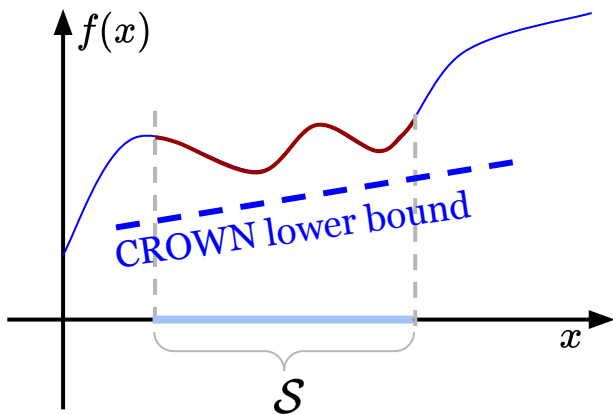
Global nonlinear optimization



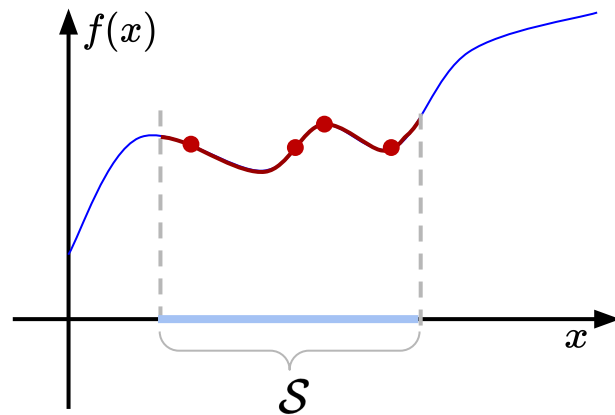
From verification to **optimization**

$$x^* = \arg \min_{x \in \mathcal{S}} f(x)$$

CROWN can find a lower bound only,
does not solve the problem directly



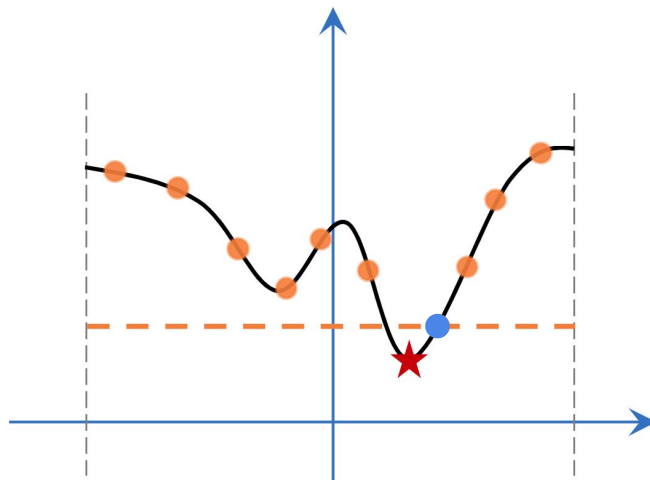
Sampling can give solutions but
can be far away from optimal



From verification to **optimization**

Let's start with randomly sampling some points

The current best solution ● gives an **upper bound** of the optimal solution ★

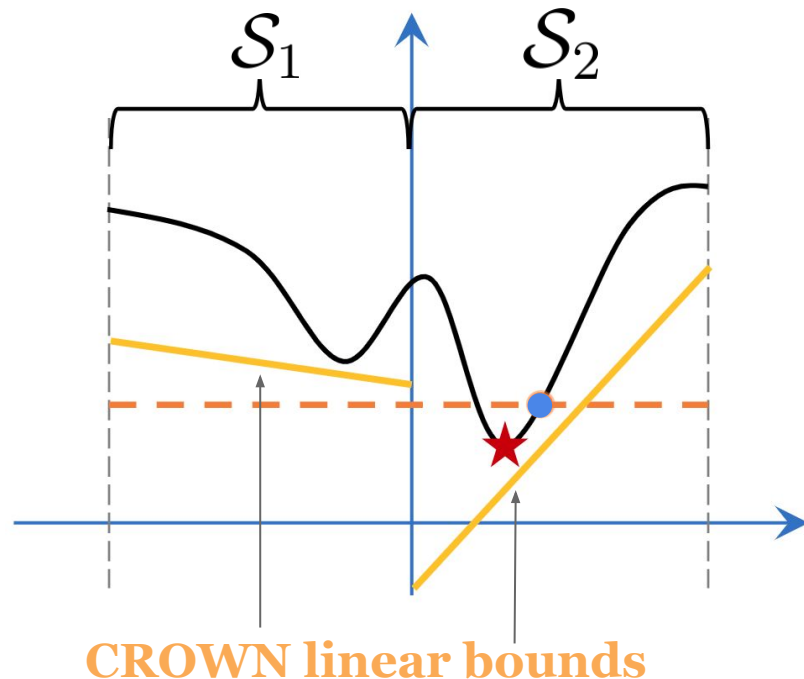


BaB-ND: Long-Horizon Motion Planning with Branch-and-Bound and Neural Dynamics.
K. Shen, J. Yu, H. Zhang, Y. Li (ICLR 2025)

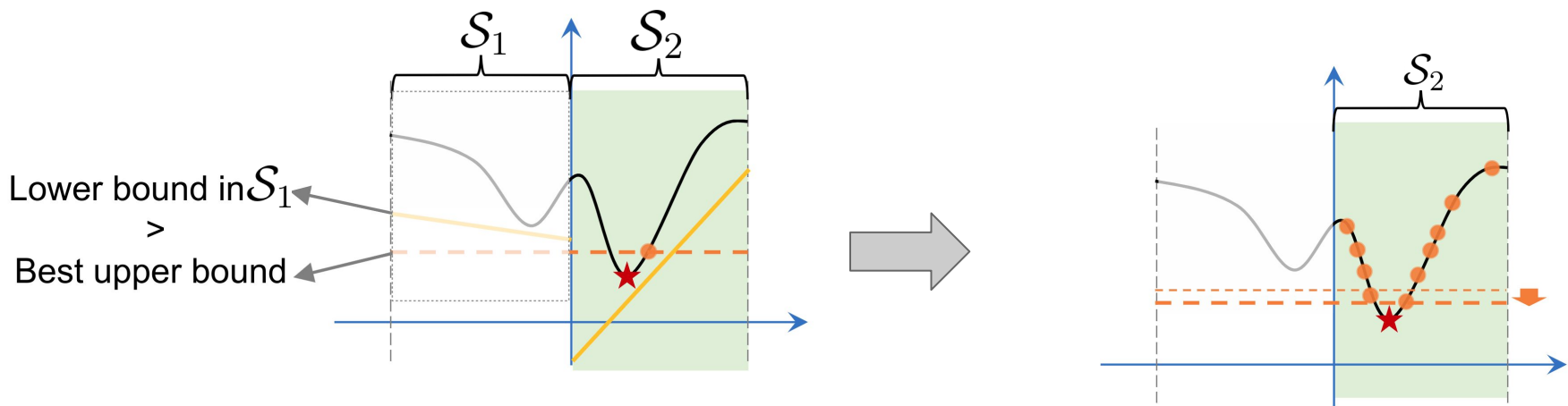
From verification to **optimization**

Branching the problem into two subproblems S_1, S_2

Bounding each subproblem (a neural network function) using **CROWN**



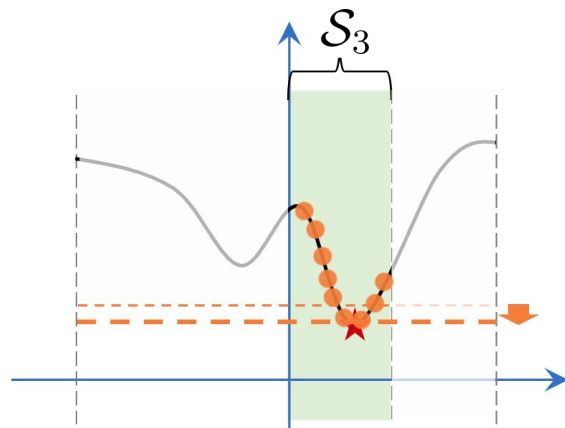
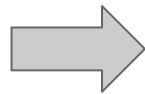
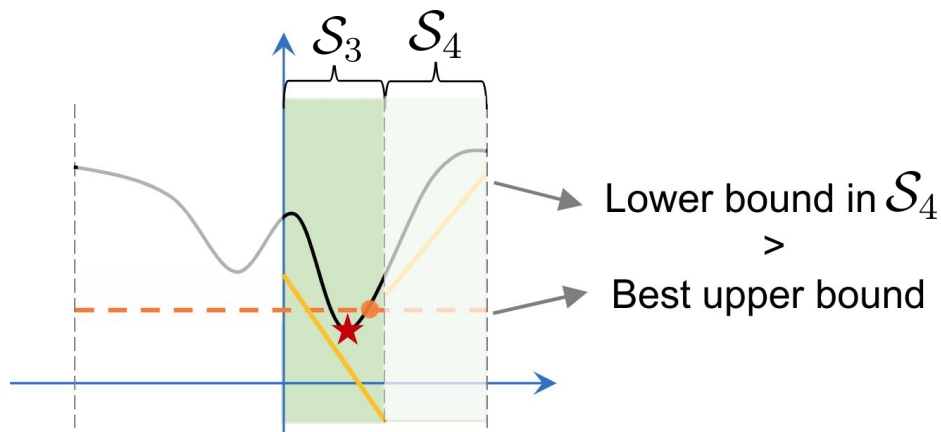
From verification to **optimization**



Key insight: With the lower bound, we remove S_1 from consideration (verification now used for **optimization**!)

Search space reduced and sample again. Solution improved

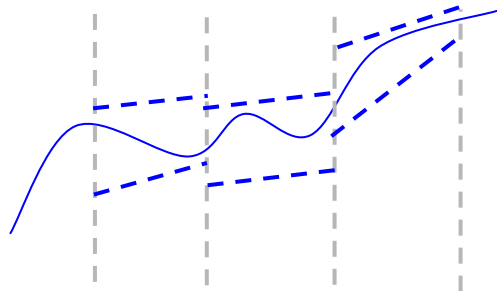
From verification to **optimization**



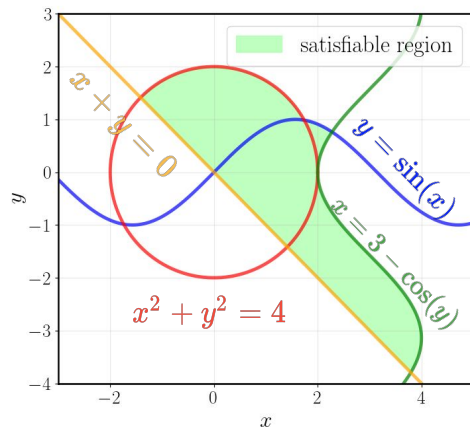
Keep going, removing S_4 using the CROWN lower bound

Very close to the optimal solution now!

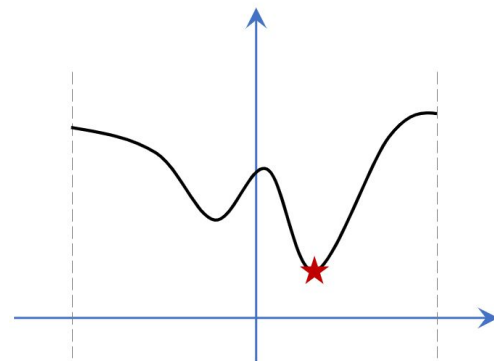
What α, β -CROWN can do for us?



Bounding nonlinear functions



Satisfiability problems with
nonlinear arithmetic



Global nonlinear optimization



α, β -CROWN is constantly being improved

Many new features and improves are currently being developed:

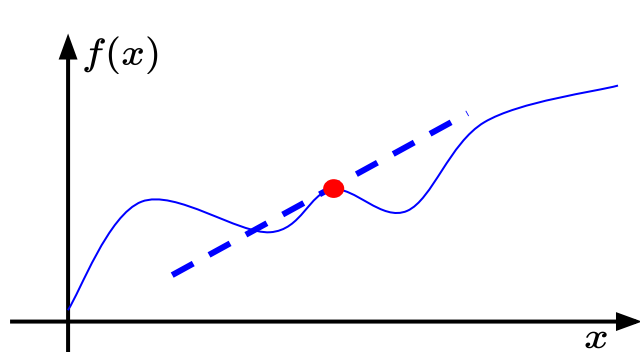
- Constrained nonlinear optimization (to be released Fall 2026)
- Satisfiability problems with integer variables & general SMT-NRA problems
- High-order computational graphs
- Bounding over differential equations
- ...

Our team is very interested in new applications of α, β -CROWN

- Looking forward to collaborations! 🤝
- In many cases, we can help with developing specialized verification algorithms and configurations to significantly improve results
- Found bugs? Report as a github issue

Up next and more resources

- Interactive coding demo (Colab) (presented by Xiangru Zhong)
 - A new Python API, much easier to use
- After the break: A guide on applying α, β -CROWN to control problems
 - Reachability analysis, Lyapunov stability, MPC
 - Also comes with a coding demo on control problems (good starting point for your own paper!)
 - dReal-compatible API - port your code directly!
- Interested in more technical backgrounds on α, β -CROWN?
 - Tutorial paper: <https://arxiv.org/pdf/2605.26577>
 - More tutorial materials at neural-network-verification.com
 - Check out UIUC ECE598HZ course slides

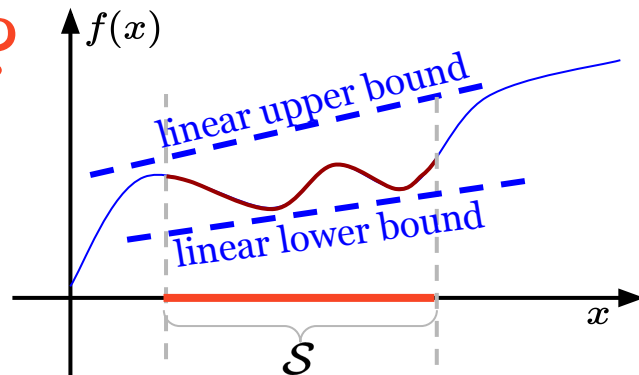


chain rule & backpropagation

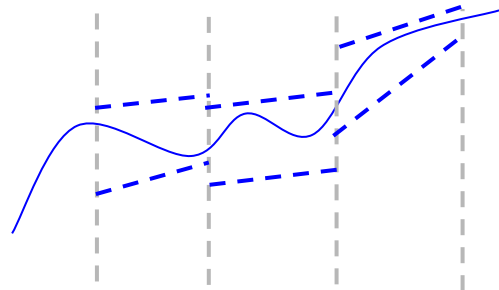
Questions?



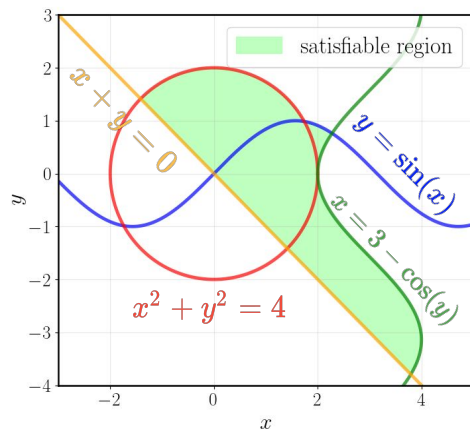
Scalable, GPU
acceleration



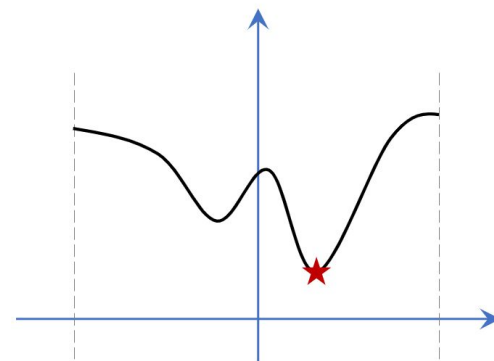
CROWN linear bound propagation



Bounding nonlinear functions



Satisfiability problems with
nonlinear arithmetic



Global nonlinear optimization

Next: coding demo

Bring your laptop and check out the coding demo!

<https://abcrown.org/tutorial>